

UNIDAD DIDÁCTICA 01

Relación 2 CRUD y copias en ficheros de texto

Ocho ejercicios progresivos sobre interpretación de registros, consultas, altas, modificaciones, bajas, validación y recuperación de datos.

Relación 2 CRUD y copias en ficheros de texto

Ejercicios correspondientes a los apartados 4 y 5 de la Unidad Didáctica 1. Los ejercicios crecen en extensión y reutilizan las operaciones de lectura y escritura ya trabajadas.

Indicaciones. No copies los ejemplos del tema: resuelve cada problema adaptando las operaciones aprendidas a un contexto diferente. Los archivos necesarios están incluidos en la carpeta `Datos` del ZIP. Trabaja siempre sobre una copia de los datos de partida.

01 Catálogo de videojuegos: interpretar registros

Apartado 4.1 · `split`, arrays y conversión de tipos

Archivo: `videojuegos.csv`

Lee `videojuegos.csv`. La primera línea es la cabecera y las siguientes contienen `id;titulo;plataforma`. Convierte cada línea de datos en sus tres campos y muéstralos con un formato distinto al original.

- Ignora la cabecera al procesar los registros.
- Separa cada línea con `split(";", -1)` y comprueba que contiene exactamente tres campos.
- Convierte el ID con `Integer.parseInt(...)` y controla un posible `NumberFormatException`.
- Muestra cada videojuego como `[101] Hollow Knight - PC`.

Comprueba: cambia temporalmente un ID por una letra y verifica que el programa informa del error sin intentar usar ese registro.

02 Buscador de alumnos por identificador

Apartado 4 · Consulta de datos en un TXT

Archivo: `alumnos.csv`

Crea un programa que busque un alumno concreto dentro de `alumnos.csv` a partir de un ID introducido por teclado.

- Lee todas las líneas del fichero en UTF-8.
- Recorre solo los registros de datos y compara el ID solicitado con el primer campo.
- Si lo encuentras, muestra nombre y grupo; si no existe, muestra un mensaje específico.
- El programa debe distinguir entre un ID introducido que no sea numérico y un ID numérico que simplemente no exista.

Comprueba: prueba con el ID 3, con el 99 y con el texto `abc`. Los tres casos deben producir resultados distintos y controlados.

03 Actualización de stock sin perder registros

Apartado 4.2 · readAllLines, ArrayList y reescritura

Archivo: inventario.csv

Trabaja con `inventario.csv`. El usuario indicará el ID de un producto y el nuevo stock. Debes modificar únicamente ese registro y conservar todos los demás exactamente una vez.

- Carga las líneas originales y construye una segunda lista para el contenido actualizado.
- Valida que el registro tenga tres campos y que ID y stock sean números enteros.
- Solo sustituye la línea cuyo ID coincida; el resto debe añadirse sin cambios a la lista nueva.
- Escribe el archivo únicamente cuando el producto solicitado exista y el nuevo stock sea válido.

Comprueba: anota el número de líneas antes y después. Modificar un producto no debe aumentar ni reducir el número de registros. Buscar un ID inexistente no debe reescribir el fichero.

04 Cancelar una reserva

Apartado 4.2 · Borrado mediante reconstrucción del fichero

Archivo: reservas.csv

Implementa la baja de una reserva en `reservas.csv`. En un fichero de texto no borrarás bytes en mitad del archivo: crearás una nueva lista con todos los registros excepto el solicitado.

- Pide el ID de la reserva que se desea cancelar.
- Conserva la cabecera del CSV y todas las reservas cuyo ID sea distinto.
- Utiliza una variable booleana para saber si realmente se encontró el registro.
- Si no existe, no escribas de nuevo el archivo. Si existe, guarda la lista resultante.

Comprueba: elimina la primera reserva, después la última y finalmente intenta borrar un ID inexistente. En ningún caso deben desaparecer registros adicionales.

05 Altas de biblioteca sin duplicados

Apartado 4.3 · Validación y control de duplicados

Archivo: biblioteca.csv

Añade nuevos libros a `biblioteca.csv` aplicando reglas que el archivo por sí mismo no puede garantizar.

- No permitas IDs repetidos.
- No permitas títulos repetidos aunque cambien mayúsculas, minúsculas o espacios al principio y al final; utiliza `trim()` y `equalsIgnoreCase(...)`.
- Rechaza ID no numérico, ID menor o igual que cero, título vacío y autor vacío.
- Solo cuando todos los datos sean válidos añade una nueva línea al final del fichero.

Comprueba: intenta añadir `id=8, titulo=" NADA "`; debe detectarse como duplicado del libro existente. Después añade un libro realmente nuevo.

06 Importador resistente de productos

Apartado 4.3 · Validar antes de aceptar datos

Archivo: productos_errores.csv

Procesa `productos_errores.csv`, que contiene filas correctas y filas defectuosas. El objetivo no es arreglar el archivo a mano, sino hacer que Java seleccione los registros utilizables.

- Valida número de campos, ID entero positivo, nombre no vacío y stock entero mayor o igual que cero.
- Detecta IDs duplicados entre los registros que ya hayas aceptado.
- Por cada fila incorrecta, muestra su número de línea y el motivo; continúa con las siguientes.
- Guarda los registros válidos en un archivo nuevo llamado `productos_validos.csv`. Nunca sobrescribas el archivo de entrada.

Comprueba: la ejecución debe terminar aunque haya varios errores. Abre `productos_validos.csv` y confirma que no contiene filas inválidas ni IDs duplicados.

07 Editar con red de seguridad

Apartado 5 · Copias de seguridad y restauración

Archivo: inventario.csv

Convierte la modificación de stock del ejercicio 3 en una operación segura. Antes de cambiar `inventario.csv`, el programa debe conservar una copia recuperable.

- Crea `datos/copias` si hace falta y copia el archivo actual a `inventario.bak` con `Files.copy(..., REPLACE_EXISTING)`.
- Realiza después la modificación solicitada.
- Añade una opción independiente para restaurar `inventario.bak`; antes de sustituir los datos actuales, pide confirmación al usuario.
- Si no existe la copia, informa y conserva el archivo actual.

Comprueba: modifica un stock, comprueba que el original anterior está en la copia y restaura. Tras restaurar, el contenido debe coincidir con el estado previo a la modificación.

08 Gestor de préstamos en fichero

Apartados 4 y 5 · CRUD, validación y copias

Crea desde cero un pequeño gestor de préstamos con el archivo `prestamos.csv`. Cada registro tendrá `id;persona;material`. El fichero puede no existir al iniciar el programa.

- Ofrece un menú para alta, listado, búsqueda por ID, cambio de material, baja, crear copia y restaurar copia.
- Impide ID repetido y persona o material vacíos. La comparación del ID se hará sobre datos ya validados.
- Las modificaciones y bajas deben reconstruir el contenido, no escribir en mitad de una línea.
- Antes de cualquier operación que reescriba el fichero, crea una copia `prestamos.bak`.
- El menú debe seguir funcionando después de un error de entrada o de buscar un ID inexistente.

Comprueba: realiza al menos dos altas, una búsqueda, una modificación, una baja, una restauración y una operación con datos erróneos. Reinicia el programa y verifica que el resultado guardado se conserva.
